SZTUCZNA INTELIGENCJA I INŻYNIERIA WIEDZY

–

LISTA 1

Zofia Różańska, 280526

25 MARCA 2026

Wstęp

Pierwszym krokiem do wykonania zadania było załadowanie i przetworzenie danych z plików w formacie GTFS. W tym celu stworzyłam klasę GTFSLoader. Metoda load_all(target_date) wczytuje pliki calendar_dates.txt, calendar.txt, routes.txt, stop_times.txt, stops.txt i trips.txt do DataFrame z biblioteki Pandas, a następnie przetwarza je filtrując dane na podstawie podanej daty target_date oraz zapewniając, że każdy stop ma swojego parent_station.

Po wstępnym przetworzeniu danych, należało zbudować graf modelujący sieć kolejową. W tym celu stworzyłam klasę GraphBuilder, która przyjmuje przetworzone ramki danych i buduje graf w postaci słownika, gdzie kluczem jest id parent_station, a wartością jest lista krawędzi reprezentujących kolejne przejazdy między stacjami połączonymi kursami pociągów w danym dniu (Edge).

Do przechowywania szczegółowych informacji o stacjach stworzyłam również klasę Stop. GraphBuilder.get_metadata() zwraca słownik, gdzie kluczem jest id parent_station, a wartością jest obiekt Stop zawierający nazwę stacji i resztę dostępnych o niej informacji.

```
class Stop:
    def __init__(self, stop_id, stop_name, lat=None, lon=None, lines=None):
        self.stop_id = stop_id
        self.stop_name = stop_name
        self.lat = lat
        self.lon = lon
        self.lines = lines if lines is not None else set()

class Edge:
    def __init__(self, to_stop, departure, arrival, trip_id, route_name):
        self.to_stop = to_stop
        self.departure = departure  # seconds from midnight
        self.arrival = arrival      # seconds from midnight
        self.trip_id = trip_id
        self.route_name = route_name
```

Zadanie 1

Algorytm Dijkstry

Aby znaleźć najlepsze połączenie ze stacji A do stacji B na podstawie kryterium czasu podróży lub liczby przesiadek, zaimplementowałam klasę Dijkstra. Operuje ona na grafie zbudowanym przez GraphBuilder. Aby zwiększyć efektywność, algorytm wyszukiwania najtańszej ścieżki zaimplementowałam z wykorzystaniem kolejki priorytetowej (min-heap) z biblioteki heapq.

Implementując algorytm, inspirowałam się tym podejściem. Mój algorytm bazuje na kodzie źródłowym, jednak musiał zostać dostosowany do specyfiki problemu. W klasycznym algorytmie Dijkstry, krawędzie są po prostu nieujemnymi wagami, jednak w tym przypadku cały stan grafu jest zależny od czasu. Aby znaleźć najlepsze połączenie kolejowe, należało przede wszystkim uwzględnić czasy odjazdów i przyjazdów pociągów, a także czas potrzebny na przesiadkę. Algorytm w każdej iteracji wybiera z kolejki priorytetowej wierzchołek o najniższym koszcie i rozważa wszystkie połączenia wychodzące z tej stacji – dla każdego połączenia sprawdza, czy można nim dojechać do sąsiedniej stacji szybciej niż dotychczas znany czas dotarcia do tej stacji. Jeśli tak, aktualizuje czas dotarcia i dodaje sąsiednią stację do kolejki priorytetowej. Użycie kolejki priorytetowej pozwala na efektywne znajdowanie najtańszej ścieżki w grafie, nawet gdy graf jest duży i złożony.

Ścieżka do pliku: src/algorithms/dijkstra.py

Przykład użycia

```
$ python main_dijkstra.py 'Zgorzelec' 'Jaworzyna Śląska' t  20260324 15:00
From Station                    | To Station                    | Line       | Departure  | Arrival
-------------------------------------------------------------------------------------------------------
Zgorzelec                       | Wrocław Główny                | D10        | 16:17:00   | 18:27:00
Wrocław Główny                  | Jaworzyna Śląska              | D6         | 18:34:00   | 19:18:00

[METRICS]
Travel Time (s): 69480
Calculation Time: 0.001063 s
```

```
$ python main_dijkstra.py 'Zgorzelec' 'Jaworzyna Śląska' p  20260324 15:00
From Station                    | To Station                    | Line       | Departure  | Arrival
-------------------------------------------------------------------------------------------------------
Zgorzelec                       | Legnica                       | D10        | 16:17:00   | 17:28:00
Legnica                         | Jaworzyna Śląska              | D91        | 19:15:00   | 20:14:00

[METRICS]
Number of Transfers: 1
Calculation Time: 0.001112 s
```

W powyższym przykładzie wyraźnie widać, że optymalizacja względem czasu i optymalizacja względem przesiadek znacząco się od siebie różnią. Przy optymalizacji liczby

przesiadek algorytm nie patrzy na to o której godzinie dotrzemy do celu. Dla niego każde rozwiązanie z 1 przesiadką będzie wystarczająco dobre – nie zwraca uwagi na to, że moglibyśmy dotrzeć do celu o godzinę szybciej.

Algorytm A*

Aby znaleźć najlepsze połączenie ze stacji A do stacji B, zaimplementowałam klasę AStar . Algorytm ten stanowi rozszerzenie Dijkstry o funkcję heurystyczną, która pozwala nadać poszukiwaniom kierunek i skupić się na wierzchołkach przybliżających nas do celu. W mojej implementacji algorytm minimalizuje funkcję kosztu $f(n) = g(n) + h(n)$, gdzie $g(n)$ to rzeczywisty koszt dotarcia do danej stacji, a $h(n)$ to szacowany koszt dotarcia ze stacji do celu.

Tak samo jak przy implementacji algorytmu Dijkstry, implementacja algorytmu A* musiała zostać dostosowana do specyfiki problemu. W klasycznym algorytmie A*, krawędzie są po prostu nieujemnymi wagami. Jednak w tym przypadku cały stan grafu jest zależny od czasu. Aby znaleźć najlepsze połączenie kolejowe, należało przede wszystkim uwzględnić czasy odjazdów i przyjazdów pociągów, a także czas potrzebny na przesiadkę.

Podobnie jak w przypadku Dijkstry, wykorzystałam kolejkę priorytetową heapq do zarządzania zbiorem wierzchołków do sprawdzenia (open_list) Kluczową różnicą jest jednak kryterium sortowania w kolejce. Zamiast samego czasu dotarcia, algorytm bierze pod uwagę sumę kosztu rzeczywistego i heurystyki. Dzięki temu rozwiązanie jest znajdowane znacznie szybciej, ponieważ algorytm w pierwszej kolejności sprawdza pociągi jadące w stronę stacji docelowej, odkładając na później te, które oddalają nas od celu.

Ścieżka do pliku: src/algorithms/astar.py

Modyfikacje poprawiające wydajność i jakość rozwiązania

- **Heurystyka dla czasu**: dla kryterium czasu przejazdu zastosowałam heurystykę opartą na przewidywanym, idealnym czasie przejazdu. Jest on liczony jako iloraz odległości obliczanej wzorem haversine podzielonej przez maksymalną prędkość pociągu (na Polskich torach standardowo 160 km/h). Dzięki temu heurystyka nie przeszacowuje kosztu, co w pozwala znaleźć optymalne rozwiązanie.
- **Heurystyka dla przesiadek:** w trybie minimalizacji przesiadek zaimplementowałam sprawdzanie spójności linii. Algorytm sprawdza, czy stacja bieżąca i docelowa obsługują te same numery linii. Jeśli nie, heurystyka zwraca wartość 1, informując, że pasażera czeka co najmniej jedna przesiadka. Pozwala to na znacznie szybsze odrzucanie ścieżek, które składają się z wielu przesiadek.
- **Złożony koszt:** Zarówno w $g(n)$ jak i $h(n)$ wykorzystałam krotki kosztów. W trybie czasu priorytetem jest czas przyjazdu, a drugim parametrem liczba przesiadek. W trybie przesiadek kolejność jest odwrotna. Dzięki temu przy identycznym czasie przyjazdu algorytm automatycznie wybierze połączenie z mniejszą liczbą przesiadek,

co znacząco poprawia jakość zwracanych tras. Analogicznie, przy identycznej liczbie przesiadek, algorytm wybierze trasę, dzięki której wcześniej dotrzemy do celu. Przed wprowadzeniem tej zmiany wynikowe trasy często zawierały dużo bezcelowych przesiadek lub wybierały nieoptymalne godziny połączeń.

Przykład użycia

```
$ python main_astar.py 'Zgorzelec' 'Jaworzyna Śląska' t  20260324 15:00
From Station                    | To Station                    | Line       | Departure  | Arrival
-------------------------------------------------------------------------------------------------------
Zgorzelec                       | Wrocław Główny                | D10        | 16:17:00   | 18:27:00
Wrocław Główny                  | Jaworzyna Śląska              | D6         | 18:34:00   | 19:18:00

[METRICS]
Travel Time (s): 69480
Calculation Time: 0.005256 s
```

```
$ python main_astar.py 'Zgorzelec' 'Jaworzyna Śląska' p  20260324 15:00
From Station                    | To Station                    | Line       | Departure  | Arrival
-------------------------------------------------------------------------------------------------------
Zgorzelec                       | Wrocław Główny                | D10        | 16:17:00   | 18:27:00
Wrocław Główny                  | Jaworzyna Śląska              | D6         | 18:34:00   | 19:18:00

[METRICS]
Number of Transfers: 1
Calculation Time: 0.002696 s
```

Aby porównać wydajność i efektywność algorytmu A* skorzystamy z wyszukiwania tego samego połączenia co w przypadku algorytmu Dijkstry. Algorytm z kryterium czasu zwrócił takie samo połączenie jak algorytm Dijkstry – jest to faktycznie optymalne rozwiązanie (potwierdzone przez wyszukiwarkę Kolei Dolnośląskich). Dla kryterium czasu przesiadek widzimy jednak pewną różnicę – algorytm A* wybrał inne, bardziej optymalne połączenie niż algorytm Dijkstry. Zawdzięczamy to kosztowi złożonemu – liczba przesiadek jest taka sama jak w wyniku Dijkstry, jednak czas dotarcia do celu jest bardziej optymalny, więc algorytm wybrał tę opcję.

Zadanie 2

Algorytm Tabu Search

Aby rozwiązać problem znalezienia optymalnej trasy przechodzącej przez wiele zadanych stacji (problem komiwojażera) zaimplementowałam klasę TabuSearch. Algorytm ten należy do metaheurystyk przeszukiwania lokalnego i pozwala na znajdowanie dobrych (nie zawsze optymalnych) rozwiązań w rozsądnym czasie. W odróżnieniu od algorytmów Dijkstry i A*, które znajdują optymalną ścieżkę między dwoma punktami, Tabu Search stara się znaleźć optymalną permutację stacji, które należy odwiedzić. Funkcja kosztu jest tutaj sumą kosztów przejazdów pomiędzy poszczególnymi stacjami z listy.

Algorytm rozpoczyna od wygenerowania losowego rozwiązania początkowego, a następnie w każdej iteracji przeszukujemy sąsiedztwo aktualnego rozwiązania w celu znalezienia lepszej trasy. Sąsiadów rozwiązania tworzymy poprzez operację swap – zamianę miejscami dwóch elementów permutacji. Kluczowym elementem algorytmu jest lista tabu, czyli lista, która przechowuje ruchy niedozwolone. Trafiają do niej niedawno wykonane ruchy, co zapobiega powracaniu do wcześniej odwiedzonych rozwiązań. Dzięki zastosowaniu listy tabu algorytm jest w stanie uciec z optimum lokalnego.

Do obliczania kosztu przejścia między kolejnymi stacjami w permutacji wykorzystałam zaimplementowany w poprzednim zadaniu algorytm A*. Aby uniknąć wielokrotnego obliczania ścieżki między tymi samymi stacjami zastosowałam mechanizm cache, który zapisuje wyniki wywołań A*. Powoduje to znaczący spadek ilości wykonanych obliczeń.

W konstruktorze klasy TabuSearch dodałam argument fixed_tabu_size, który pozwala wybrać, czy lista tabu ma mieć ustaloną pojemność, czy ma być nieograniczona. Jeśli ma mieć ustaloną pojemność, jest ona obliczana wprost proporcjonalnie do długości listy stacji do odwiedzenia.

Zaimplementowałam także kryterium aspiracji, które pozwala zaakceptować ruch znajdujący się na liście tabu, jeśli jest on naprawdę optymalny. Dzięki temu algorytm nie blokuje potencjalnie najlepszych rozwiązań.

W każdej iteracji stosujemy próbkowanie sąsiedztwa – rozważamy tylko losową próbkę sąsiedztwa. Pozwala to znacząco przyspieszyć działanie algorytmu, jednak grozi to potencjalną utratą optymalnego rozwiązania.

Ścieżka do pliku: src/algorithms/tabu.py

Przykład użycia

```
$ python main_tabu.py 'Zgorzelec' 'Legnica; Jelenia Góra; Sobótka; Dzierżoniów Śląski' t 20260324 11:30
From Station                | To Station                  | Line       | Departure  | Arrival
-----------------------------------------------------------------------------------------------------
Zgorzelec                   | Jelenia Góra                | D62        | 11:44:00   | 13:15:00
Jelenia Góra                | Jaworzyna Śląska            | D6         | 13:27:00   | 14:56:00
Jaworzyna Śląska            | Legnica                     | D90/D91    | 15:01:00   | 15:50:00
Legnica                     | Jaworzyna Śląska            | D91        | 16:01:00   | 16:49:00
Jaworzyna Śląska            | Dzierżoniów Śląski          | D40        | 16:53:00   | 17:27:00
Dzierżoniów Śląski          | Wrocław Główny              | D40        | 18:13:00   | 19:37:00
Wrocław Główny              | Zgorzelec                   | D10        | 21:45:00   | 23:53:00

[METRICS]
Travel Time (s): 44580
Calculation Time: 0.292812 s
```

```
$ python main_tabu.py 'Zgorzelec' 'Legnica; Jelenia Góra; Sobótka; Dzierżoniów Śląski' p 20260324 11:30
From Station                | To Station                  | Line       | Departure  | Arrival
-----------------------------------------------------------------------------------------------------
Zgorzelec                   | Jelenia Góra                | D62        | 11:44:00   | 13:15:00
Jelenia Góra                | Wrocław Główny              | D6         | 13:27:00   | 15:38:00
Wrocław Główny              | Kobierzyce                  | D8/D4/D91  | 16:40:00   | 17:03:00
Kobierzyce                  | Dzierżoniów Śląski          | D4         | 17:52:00   | 18:57:00
Dzierżoniów Śląski          | Legnica                     | D91        | 18:59:00   | 20:36:00
Legnica                     | Zgorzelec                   | D10        | 22:43:00   | 23:53:00

[METRICS]
Number of Transfers: 5
Calculation Time: 0.058412 s
```

```
$ python main_tabu.py 'Zgorzelec' 'Legnica; Jelenia Góra; Sobótka; Dzierżoniów Śląski' t 20260324 11:30
Connection not found.

[METRICS]
Travel Time (s): inf
Calculation Time: 0.185018 s
```

Algorytm Tabu Search ma charakter heurystyczny i częściowo losowy, przez co nie gwarantuje znalezienia optymalnego rozwiązania przy każdym uruchomieniu, w przeciwieństwie do algorytmów Dijkstry i A*. Przy kilku próbach uruchomienia tej samej komendy, raz na jakiś czas zdarzy się, że algorytm nie znajdzie rozwiązania. W mojej implementacji losowość pojawia się w dwóch miejscach: podczas generowania rozwiązania początkowego i podczas próbkowania sąsiedztwa. Czasami zdarza się, że wylosowane rozwiązania nie pozwalają na znalezienie optymalnego rozwiązania. W moim przypadku algorytm nie zwraca żadnego rozwiązania, ponieważ przy tworzeniu grafu pobieram tylko połączenia w wybranym dniu. Gdyby dodać do grafu połączenia z kilku dni, algorytm znalazłby prawdopodobnie nieoptymalną trasę rozciągającą się na więcej niż jedną dobę.

Podsumowanie

W projekcie zaimplementowałam algorytmy Dijkstry, A* oraz Tabu Search do wyznaczania tras w dynamicznym grafie kolejowym. Głównym wyzwaniem było przejście z teoretycznych algorytmów uwzględniających jedynie niezmienną wagę krawędzi, na ich rozszerzenia uwzględniające godziny odjazdów i przyjazdów oraz przesiadki. Jednak najbardziej czasochłonnym etapem projektu było samo przetwarzanie danych GTFS i budowanie grafu, na którym operują algorytmy, ponieważ wymagało to bardzo dokładnego zrozumienia problemu. Podczas testowania rozwiązań porównywałam moje wyniki z wynikami wyszukiwania trasy na stronie Kolei Dolnośląskich.

Algorytmy Dijkstry i A* przy każdym uruchomieniu dawały takie same wyniki, ponieważ są to algorytmy deterministyczne. Jednak A* z heurystykami i ulepszeniami w ogólnym przypadku dawała lepsze wyniki. Z kolei Tabu Search, który zawiera w sobie pewną losowość, zwracał różne wyniki przy kolejnych uruchomieniach tej samej komendy.

W trakcie pracy nad kodem korzystałam głównie z materiałów załączonych w instrukcji. Dodatkowo, korzystałam również z LLM (Gemini) w celu debugowania błędów, weryfikacji poprawności implementacji oraz w celu napisania funkcji testujących kod i formatujących wyniki. Ostatecznie projekt pokazał, że w starciu z rzeczywistymi danymi nawet teoretycznie optymalne algorytmy wymagają szeregu modyfikacji, aby zwracane przez nie wyniki były użyteczne i optymalne.